

Contents

1	Ant Stack Implementation Appendix	1
1.1	Introduction	1
1.2	Layer-by-Layer Integration	2
1.2.1	AntBody Layer: Physical Simulation and Sensing	2
1.2.2	AntBrain Layer: Neural Architecture	3
1.2.3	AntMind Layer: Cognitive Modeling	4
1.3	Species-Specific Implementations	5
1.3.1	Formica Species Configuration	5
1.3.2	Camponotus Species Configuration	6
1.4	Evaluation and Benchmarking	7
1.4.1	Navigation Performance Metrics	7
1.4.2	Robustness Testing	8
1.5	Implementation Workflow	9
1.5.1	Development Pipeline	9
1.5.2	Code Organization	9
1.6	Integration Benefits	10
1.6.1	Reproducibility	10
1.6.2	Extensibility	10
1.6.3	Validation	10
1.7	Future Directions	10
1.7.1	Advanced Learning Mechanisms	10
1.7.2	Hardware Integration	10
1.7.3	Biological Validation	10
1.8	Conclusion	11

1 Ant Stack Implementation Appendix

1.1 Introduction

This appendix provides a comprehensive mapping of how the cohereAnts research framework can be implemented within “The Ant Stack” computational architecture. The Ant Stack, as described by Friedman (2025), offers a modular, three-layer framework for emulating ant intelligence from physical embodiment to collective cognition. Here we demonstrate how our vibrational theory of olfaction, spectroscopic analysis, and behavioral modeling can be systematically integrated into this standardized computational framework.

Implementation Strategy: We map cohereAnts modules to Ant Stack layers using explicit I/O contracts, ensuring reproducible experiments while maintaining the biological plausibility of our theoretical framework. The examples below are thin adapters

that call `src/` functions; no scientific logic resides in these adapters.

1.2 Layer-by-Layer Integration

1.2.1 *AntBody Layer: Physical Simulation and Sensing*

Sensilla Morphology Integration The `cohereAnts` sensilla analysis module maps directly to `AntBody`'s morphological modeling:

```
1 # AntBody sensilla configuration
2 class AntBodySensilla:
3     def __init__(self, species_preset: str):
4         # Load species-specific sensilla parameters
5         self.lengths = load_sensilla_lengths(species_preset)
6         self.diameters = load_sensilla_diameters(species_preset)
7         self.optimal_wavelengths = self._calculate_resonance()
8
9     def _calculate_resonance(self):
10        # Implement cohereAnts resonance theory
11        return self.lengths * 4 # Quarter-wavelength resonance
        (delegated to src/sensilla in production)
```

I/O Contract: - **Observations:** Sensilla dimensions (m), resonance frequencies (THz), quality factors - **Actions:** Antenna positioning, sensilla orientation - **Physics:** 1 kHz update rate, contact dynamics for substrate interaction

Spectroscopy and Atmospheric Transmission Integration of `cohereAnts` atmospheric transmission models:

```
1 class AntBodySpectroscopy:
2     def __init__(self, environment_preset: str):
3         self.transmission_curves = load_atmospheric_data(
4             environment_preset)
5         self.spectral_resolution = 0.01 # m
6
7     def get_transmission(self, wavelength: float, distance:
8         float) -> float:
9         # Implement cohereAnts atmospheric transmission model
10        return calculate_atmospheric_transmission(wavelength,
11            distance) # from src/core
```

Configuration Parameters: - Atmospheric windows: 2-5 m, 8-14 m, 17-25 m -

Transmission coefficients: 0.7-0.9 for optimal windows - Distance-dependent attenuation models

1.2.2 AntBrain Layer: Neural Architecture

Olfactory Processing Pipeline Mapping cohereAnts vibrational theory to AntBrain's AL→MB→CX architecture:

```
1 class AntBrainOlfaction:
2     def __init__(self, neuron_count: int = 100000):
3         # Antennal Lobe (AL) - odor coding
4         self.al_neurons = self._initialize_al_circuit()
5         # Mushroom Body (MB) - associative learning
6         self.mb_neurons = self._initialize_mb_circuit()
7         # Central Complex (CX) - spatial integration
8         self.cx_neurons = self._initialize_cx_circuit()
9
10    def _initialize_al_circuit(self):
11        # Implement cohereAnts vibrational detection theory (
12            # delegates to src components in production)
13        # Each glomerulus responds to specific molecular
14            # vibrations
15        return VibrationalGlomeruliCircuit()
16
17    def _initialize_mb_circuit(self):
18        # Kenyon cells for odor-memory associations
19        return KenyonCellCircuit()
20
21    def _initialize_cx_circuit(self):
22        # Ring attractor for heading representation
23        return RingAttractorCircuit()
```

Neural Implementation Details: - **AL Layer:** 50 glomeruli, each tuned to specific vibrational frequencies - **MB Layer:** 2500 Kenyon cells with sparse coding (5% activity) - **CX Layer:** 16-heading ring attractor with 100 neurons per heading

Vibrational Detection Circuit Implementation of cohereAnts electromagnetic theory:

```
1 class VibrationalGlomeruliCircuit:
2     def __init__(self):
3         self.frequency_tuning = np.linspace(2, 25, 50) # m
4             # to THz
5         self.quality_factors = np.ones(50) * 100
```

```

5
6     def process_spectral_input(self, spectral_data: np.ndarray)
       -> np.ndarray:
7         # Implement cohereAnts resonance detection
8         responses = np.zeros(50)
9         for i, freq in enumerate(self.frequency_tuning):
10             responses[i] = self._calculate_vibrational_response
               (spectral_data, freq)
11         return responses
12
13     def _calculate_vibrational_response(self, spectrum: np.
       ndarray,
14                                         resonant_freq: float) ->
                                               float:
15         # Apply cohereAnts electromagnetic coupling model (
               placeholder; call src functions in production)
16         coupling_strength = self._calculate_coupling(spectrum,
               resonant_freq)
17         return coupling_strength * self.quality_factors[i]

```

1.2.3 AntMind Layer: Cognitive Modeling

Active Inference for Olfactory Search Integration of cohereAnts behavioral models with active inference:

```

1 class AntMindOlfaction:
2     def __init__(self):
3         self.generative_model = self._build_olfactory_model()
4         self.policy_horizon = 2.0 # seconds
5
6     def _build_olfactory_model(self):
7         # Implement cohereAnts behavioral predictions
8         return OlfactoryGenerativeModel()
9
10    def select_policy(self, current_state: Dict) -> np.ndarray:
11        # Active inference policy selection
12        expected_free_energy = self._calculate_efe()
13        return self._minimize_free_energy(expected_free_energy)
14
15    def _calculate_efe(self) -> Dict[str, float]:
16        # Decompose into epistemic and pragmatic value

```

```

17         return {
18             'epistemic': self._calculate_epistemic_value(),
19             'pragmatic': self._calculate_pragmatic_value()
20         }

```

Stigmergy for Trail Following Implementation of cohereAnts pheromone dynamics:

```

1 class AntMindStigmergy:
2     def __init__(self):
3         self.pheromone_field = np.zeros((100, 100))
4         self.decay_rate = 0.01
5         self.diffusion_coefficient = 0.1
6
7     def update_pheromone_field(self, deposits: List[Tuple[int,
8         int, float]]):
9         # Implement cohereAnts pheromone diffusion model
10        for x, y, amount in deposits:
11            self.pheromone_field[x, y] += amount
12
13        # Apply diffusion and decay
14        self.pheromone_field = self._diffuse_and_decay()
15
16    def _diffuse_and_decay(self) -> np.ndarray:
17        # Fick's law implementation from cohereAnts
18        laplacian = self._calculate_laplacian(self.
19            pheromone_field)
20        diffusion = self.diffusion_coefficient * laplacian
21        decay = -self.decay_rate * self.pheromone_field
22        return self.pheromone_field + diffusion + decay

```

1.3 Species-Specific Implementations

1.3.1 Formica Species Configuration

```

1 # Formica species preset for Ant Stack
2 FORMICA_PRESET = {
3     'body': {
4         'sensilla_lengths': [15.2, 18.7, 22.1, 19.8, 16.5], #
5             m
6         'sensilla_diameters': [2.1, 2.8, 3.2, 2.9, 2.3], #
7             m

```

```

6      'optimal_wavelengths': [60.8, 74.8, 88.4, 79.2, 66.0],
      #   m
7      'antenna_length': 2.5,   # mm
8      'leg_count': 6,
9      'body_mass': 0.015   # g
10     },
11     'brain': {
12         'al_glomeruli_count': 50,
13         'mb_kenyon_cells': 2500,
14         'cx_heading_resolution': 16,
15         'spiking_threshold': 0.1,
16         'learning_rate': 0.01
17     },
18     'mind': {
19         'policy_horizon': 2.0,   # seconds
20         'pheromone_decay': 0.01,
21         'diffusion_coefficient': 0.1,
22         'exploration_rate': 0.2
23     }
24 }

```

1.3.2 *Camponotus* Species Configuration

```

1 # Camponotus species preset for Ant Stack
2 CAMPONOTUS_PRESET = {
3     'body': {
4         'sensilla_lengths': [22.5, 28.1, 31.7, 26.8, 24.3],   #
5         #   m
6         'sensilla_diameters': [3.2, 4.1, 4.8, 4.2, 3.6],     #
7         #   m
8         'optimal_wavelengths': [90.0, 112.4, 126.8, 107.2,
9         97.2], #   m
10        'antenna_length': 3.8,   # mm
11        'leg_count': 6,
12        'body_mass': 0.045   # g
13    },
14    'brain': {
15        'al_glomeruli_count': 60,
16        'mb_kenyon_cells': 3000,
17        'cx_heading_resolution': 20,

```

```

15         'spiking_threshold': 0.08,
16         'learning_rate': 0.015
17     },
18     'mind': {
19         'policy_horizon': 2.5, # seconds
20         'pheromone_decay': 0.008,
21         'diffusion_coefficient': 0.12,
22         'exploration_rate': 0.15
23     }
24 }

```

1.4 Evaluation and Benchmarking

1.4.1 Navigation Performance Metrics

```

1 class AntStackEvaluator:
2     def __init__(self, test_scenarios: List[str]):
3         self.scenarios = test_scenarios
4         self.metrics = {}
5
6     def evaluate_navigation(self, ant_stack: AntStack) -> Dict[
7         str, float]:
8         results = {}
9         for scenario in self.scenarios:
10             if scenario == 'trail_following':
11                 results[scenario] = self.
12                     _evaluate_trail_following(ant_stack)
13             elif scenario == 'food_search':
14                 results[scenario] = self._evaluate_food_search(
15                     ant_stack)
16             elif scenario == 'nest_return':
17                 results[scenario] = self._evaluate_nest_return(
18                     ant_stack)
19         return results
20
21     def _evaluate_trail_following(self, ant_stack: AntStack) ->
22         float:
23         # Implement cohereAnts trail following metrics (calls
24             src/behavioral metrics)
25         trail_deviation = self._calculate_trail_deviation()

```

```

20         pheromone_detection = self.
           _calculate_pheromone_detection()
21         return self._combine_metrics([trail_deviation,
           pheromone_detection])
22
23     def _evaluate_food_search(self, ant_stack: AntStack) ->
           float:
24         # Implement cohereAnts search efficiency metrics (calls
           src/behavioral metrics)
25         search_time = self._measure_search_time()
26         energy_efficiency = self._calculate_energy_efficiency()
27         return self._combine_metrics([search_time,
           energy_efficiency])

```

1.4.2 Robustness Testing

```

1 class RobustnessTester:
2     def __init__(self):
3         self.noise_levels = [0.01, 0.05, 0.1, 0.2]
4         self.adversary_types = ['sensor_noise', '
           pheromone_contamination', 'path_obstruction']
5
6     def test_noise_robustness(self, ant_stack: AntStack) ->
           Dict[str, float]:
7         results = {}
8         for noise_level in self.noise_levels:
9             performance = self._run_noisy_scenario(ant_stack,
           noise_level)
10            results[f'noise_{noise_level}'] = performance
11        return results
12
13    def test_adversary_robustness(self, ant_stack: AntStack) ->
           Dict[str, float]:
14        results = {}
15        for adversary in self.adversary_types:
16            performance = self._run_adversarial_scenario(
           ant_stack, adversary)
17            results[f'adversary_{adversary}'] = performance
18        return results

```


1.5 Implementation Workflow

1.5.1 Development Pipeline

1. **Module Mapping:** Identify cohereAnts functions for Ant Stack integration
2. **I/O Contract Definition:** Establish standardized interfaces between layers
3. **Species Preset Creation:** Develop parameterized configurations
4. **Testing Framework:** Implement evaluation metrics and benchmarks
5. **Documentation:** Create implementation guides and examples

1.5.2 Code Organization

```
1 ant_stack_cohereants/  
2     antbody/  
3         sensilla_physics.py      # cohereAnts  
4     vibrational theory  
5         spectroscopy_sensors.py  # atmospheric  
6     transmission models  
7         morphology_models.py     # species-specific  
8     parameters  
9     antbrain/  
10         olfactory_circuits.py    # A L MBCX  
11     implementation  
12         vibrational_detection.py # electromagnetic  
13     coupling  
14         learning_mechanisms.py   # STDP and plasticity  
15     antmind/  
16         olfactory_inference.py   # active inference  
17     models  
18         stigmergy_models.py      # pheromone dynamics  
19         behavioral_policies.py    # search and  
20     navigation  
21     presets/  
22         formica_config.py        # Formica species  
23     preset  
24         camponotus_config.py     # Camponotus species  
25     preset  
26         custom_species.py        # Template for new  
27     species  
28     evaluation/  
29         navigation_tests.py      # Trail following,  
30     search
```

```

20         robustness_tests.py      # Noise, adversary
    testing
21         performance_metrics.py   # Standardized
    benchmarks

```

1.6 Integration Benefits

1.6.1 Reproducibility

- **Standardized I/O:** All experiments use consistent interfaces
- **Version Pinning:** Dependencies and parameters are explicitly tracked
- **Seed Management:** Reproducible random number generation
- **Artifact Tracking:** Complete experiment provenance

1.6.2 Extensibility

- **Species Presets:** Easy addition of new ant species
- **Module Swapping:** Interchangeable components across layers
- **Parameter Tuning:** Systematic exploration of parameter space
- **Benchmark Addition:** New evaluation scenarios

1.6.3 Validation

- **Biological Plausibility:** Grounded in empirical data
- **Performance Metrics:** Quantified success criteria
- **Robustness Testing:** Resilience to real-world challenges
- **Cross-Species Transfer:** Generalization across taxa

1.7 Future Directions

1.7.1 Advanced Learning Mechanisms

- **Meta-Learning:** Adaptation across different environments
- **Collective Intelligence:** Emergent behaviors in colonies
- **Transfer Learning:** Knowledge transfer between species

1.7.2 Hardware Integration

- **Robotic Platforms:** Physical ant-inspired robots
- **Sensor Networks:** Distributed environmental monitoring
- **Edge Computing:** Efficient on-device processing

1.7.3 Biological Validation

- **Field Studies:** Comparison with natural ant behavior
- **Neural Recording:** Validation against biological data
- **Evolutionary Analysis:** Phylogenetic patterns in behavior

1.8 Conclusion

The integration of cohereAnts research into the Ant Stack framework provides a robust, reproducible platform for studying ant intelligence. By mapping our vibrational theory of olfaction, spectroscopic analysis, and behavioral modeling to the standardized three-layer architecture, we create a comprehensive system that bridges theoretical insights with computational implementation.

This implementation enables systematic exploration of ant behavior across species, environments, and experimental conditions while maintaining the biological plausibility that underpins our research. The modular design facilitates both hypothesis testing in myrmecology and applications in swarm robotics, cognitive security, and AI alignment.

Key Contributions: 1. **Systematic Integration:** Methodical mapping of cohereAnts to Ant Stack layers 2. **Species Parameterization:** Reproducible configurations for multiple ant taxa 3. **Evaluation Framework:** Standardized metrics and robustness testing 4. **Implementation Workflow:** Clear development pipeline and code organization 5. **Future Roadmap:** Extensibility and validation pathways

The resulting framework serves as a bridge between theoretical entomology and computational neuroscience, enabling reproducible research that advances our understanding of both natural ant intelligence and artificial intelligence systems.